



🛡️ Agregar el Campo "Saldo" a la Tabla Persona y Actualizar la Entidad en Spring Boot

Introducción

En esta lección, vamos a realizar **modificaciones estructurales** en nuestra aplicación para preparar el manejo de **saldos** de los clientes.

Trabajaremos en dos áreas principales:

- Agregar un nuevo campo llamado **saldo** en la **tabla persona** de nuestra base de datos.
- Actualizar nuestra **clase de entidad Persona** para incluir este nuevo campo y garantizar que no sea nulo.

¡Comencemos! 🚀

Modificaciones paso a paso

Modificando la base de datos

Abrimos **MySQL Workbench** y seguimos estos pasos:

- Conectamos a nuestra base de datos `test`.
- Localizamos la tabla `persona`.
- Damos clic derecho → **Alter Table**.

Agregamos un nuevo campo:

- **Nombre:** `saldo`
- **Tipo:** `DOUBLE`

Luego damos:

- **Apply** → **Apply nuevamente** → **Finish**.

✅ Esto agregará exitosamente la columna `saldo` a nuestra tabla `persona`.

Insertar valores de prueba en la columna `saldo`

Ahora agregamos algunos valores para probar el nuevo campo:

- Para el primer registro: 100
- Para el segundo registro: 150
- Para el tercer registro: 200

Estos son datos de ejemplo para poder trabajar en nuestras pruebas posteriores.

Result Grid						
Filter Rows:				Edit:	Export/Imp	
	id_persona	nombre	apellido	email	telefono	saldo
1	1	Juan	Perez	jperez@mail.com	5513445567	100
2	2	Karla	Gomez	kgomez@mail.com	7713445678	150
3	3	Maria	Gutierrez	mgutierrez@mail.com	88991199	200
»	NULL	NULL	NULL	NULL	NULL	NULL

Corrección adicional en la tabla `rol`

En caso necesario, aprovechamos para **corregir la asignación de roles**:

- En la tabla `rol`, aseguramos que:
 - El usuario **admin** (ID 1) tenga **rol de admin** y **rol de user**.
 - El usuario **user** (ID 2) tenga **solo rol de user**.

Aplicamos los cambios con:

- Apply** → **Apply nuevamente** → **Finish**.

✅ Con esto garantizamos que las pruebas de roles se realicen de manera adecuada.

Result Grid			
Filter Rows:			
	id_rol	nombre	id_usuario
▶	1	ROLE_ADMIN	1
	2	ROLE_USER	1
	3	ROLE_USER	2
	NULL	NULL	NULL

 **Archivo:** `src/main/java/com/example/domain/Persona.java`

Ahora modificamos la clase **Persona.java** agregando el nuevo atributo `saldo`.

Este es el código completo:

```
package gm.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.io.Serializable;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;

@Entity
@Table(name="persona")
public class Persona implements Serializable{

    private static final long serialVersionUID = 1L;
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long idPersona;

@NotEmpty
private String nombre;

@NotEmpty
private String apellido;

@NotEmpty
@email
private String email;

private String telefono;

@NotNull
private Double saldo;

public Persona() {
}

public Persona(Long idPersona, String nombre, String apellido, String email, String telefono,
    Double saldo) {
    this.idPersona = idPersona;
    this.nombre = nombre;
    this.apellido = apellido;
    this.email = email;
    this.telefono = telefono;
    this.saldo = saldo;
}

public Long getIdPersona() {
    return idPersona;
}

public void setIdPersona(Long idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
```

```
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTelefono() {
        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    public Double getSaldo() {
        return saldo;
    }

    public void setSaldo(Double saldo) {
        this.saldo = saldo;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido
            + ", email=" + email + ", telefono=" + telefono + ", saldo=" + saldo + '}';
    }
}
```

Explicación:

- Agregamos el atributo `private Double saldo;`.
 - Le aplicamos la anotación `@NotNull`, ya que el saldo **no puede ser nulo**.
 - Importamos la anotación `jakarta.validation.constraints.NotNull`.
 - No utilizamos `@NotEmpty`, ya que `@NotEmpty` es para cadenas (`String`) y no para tipos numéricos.
-

¿Cómo probarlo?

- Si tu proyecto ya está ejecutándose, **reinicia** la aplicación.
 - Verifica que **no haya errores** en la consola relacionados con la entidad `Persona`.
 - A partir de ahora, cuando trabajemos con personas, podremos ver o utilizar el nuevo campo **saldo**.
 - No tenemos ningún cambio visual de momento, sólo la aplicación debe seguir ejecutándose correctamente.
-

Conclusión

✅ En esta lección mejoramos la estructura de nuestro proyecto:

- **Agregando el campo saldo** en la base de datos y en la entidad `Persona`.
- **Asegurando integridad** usando la validación `@NotNull`.
- Dejando preparada nuestra aplicación para manejar operaciones relacionadas con **salDOS** de clientes.

¡Muy bien hecho! 🌟 ¡Seguimos avanzando de manera profesional en nuestro proyecto! 🚀

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)